DARIC: A Position-Independent KV Cache Reuse System with Distribution-Aware Recomputation

Weikang Wang
Intelligent Computing Infrastructure Lab, Lenovo Research
Beijing, China
wangwk12@lenovo.com

Haiyang Liu
Intelligent Computing Infrastructure Lab, Lenovo Research
Beijing, China
liuhy50@lenovo.com

Xin Zhou
Intelligent Computing Infrastructure Lab, Lenovo Research
Beijing, China
zhouxin16@lenovo.com

Jun Liu
Intelligent Computing Infrastructure Lab, Lenovo Research
Beijing, China
liujun8@lenovo.com

Weifeng Zhang
Intelligent Computing Infrastructure Lab, Lenovo Research
Beijing, China
weifengz@lenovo.com

arXiv:2611.09109v1 [cs.DC] 6 Nov 2026

Abstract

The context lengths of large language models (LLMs) are rapidly increasing in real-world applications such as agentic systems and retrieval-augmented generation (RAG). To accelerate LLM inference in these scenarios, reusing position-independent key-value (KV) caches across requests has emerged as a promising approach. This technique typically relies on recomputing a selected subset of tokens to restore cross-chunk interactions. However, existing solutions suffer from critical limitations in both algorithmic efficiency and system deployment. Algorithmically, they overlook the distribution of KV cache deviations, leading to either high token-selection overhead or degraded generation quality. At the system level, the reliance on custom LLM execution flows for recomputation complicates deployment and prevents seamless integration with highly optimized inference engines. In this paper, we propose DARIC (**D**istribution-**A**ware **R**ecomputation for **I**ndependent KV **C**ache Reuse), a novel KV cache reuse method based on system-algorithm co-design. At its core, DARIC introduces a distribution-aware token selection strategy that adaptively schedules token recomputation based on empirical KV deviation patterns, striking an optimal balance between generation quality and computational efficiency. Moreover, DARIC is built on a lightweight system architecture that enables independent KV cache generation and recomputation with minimal integration overhead, fully preserving existing system-level optimizations of standard inference engines. Comprehensive experiments across various datasets and LLMs demonstrate that DARIC consistently outperforms state-of-the-art methods, achieving a 1.1×–4.5× speedup in time-to-first-token (TTFT) with negligible loss in generation quality. Code is available at https://github.com/Lenovo-ICI/DARIC.

Keywords

Large Language Model, Retrieval-Augmented Generation, Position-Independent KV Cache Reuse, Partial Recomputation

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across a wide range of applications [1, 4, 5, 29]. However, driven by the growing demand for extended contexts in scenarios like agentic systems [7, 16, 41] and retrieval-augmented generation (RAG) [8, 12, 17], LLM inference faces severe bottlenecks in both computational latency and memory footprint. To address these challenges, recent works have focused on reusing key-value (KV) caches across requests.

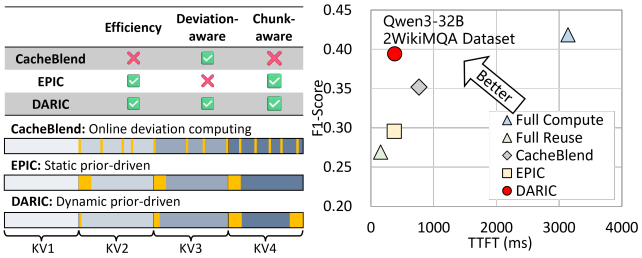


Figure 1: Comparison of partial recomputation algorithms (left) and their efficiency-quality trade-offs (right).

During LLM inference, cross-request KV cache reuse primarily occurs during the prefill stage, where the model processes the entire input prompt to generate the first token. Early approaches typically rely on strict prefix matching, sharing KV caches only if the current prompt shares an identical initial sequence with previous requests [21, 23, 44]. However, this strategy suffers from low cache hit rates in highly dynamic applications. In RAG, for instance, prompt structures often vary across requests despite retrieving the same underlying knowledge chunks. A similar issue arises in agentic systems. Although a long system prompt is commonly used, it typically embeds user-specific or project-specific information. Variations in these small but dynamic segments prevent the system prompt from being prefix-aligned across different sessions. To overcome these limitations, recent works have proposed independent KV cache reuse strategies [2, 15, 27, 28, 40]. These methods compute and store KV caches at the chunk level, enabling flexible reuse regardless of a chunk's absolute position within the prompt.

To mitigate the output quality degradation caused by position-independent reuse, recent methods typically restore cross-chunk attention via either *model fine-tuning* or *KV cache partial recomputation*. The former fine-tunes LLMs to accommodate independently generated KV caches [27, 28], which incurs prohibitive costs for

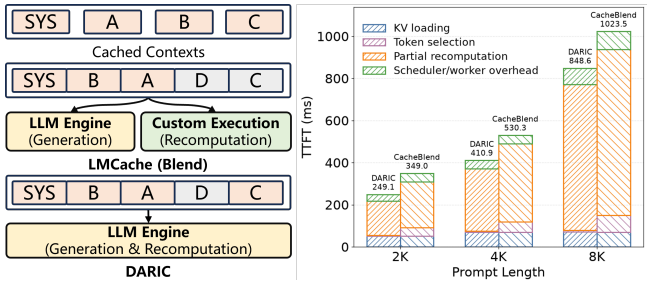


Figure 2: System-level comparison of KV cache reuse methods (left) and their TTFT breakdowns under a fixed recomputation ratio of 15% (right).

real-world deployment. The latter focuses on adapting the reused KV caches during inference [2, 15, 40], where the KV caches for a small subset of critical tokens are recomputed to restore inter-chunk correlations. While more practical in deployment, existing partial recomputation methods still suffer from significant limitations at both the algorithmic and system levels.

Algorithm-level limitations. As illustrated in Figure 1 (left), existing token selection strategies fall into two main categories: (1) online deviation-based methods [2, 40], which dynamically select a fixed ratio of tokens by calculating the deviation between independent and standard prefix-based KV caches; and (2) prior-driven methods [15], which rely on predefined rules derived from general structural heuristics. However, online KV-deviation computation introduces significant runtime overhead. In addition, selecting a fixed ratio of tokens for recomputation is inefficient when the prompt contains only a small number of long independent chunks. We observe that KV deviations correlate strongly with the number of independent chunks, implying that extensive recomputation is unnecessary in such cases. On the other hand, existing prior-driven methods are often based on overly simplistic fixed rules that do not adequately reflect the KV deviation patterns of different datasets and LLMs, which can result in suboptimal generation quality.

System-level limitations. Integrating independent KV cache reuse into existing inference engines remains fundamentally challenging. As illustrated in Figure 2 (left), the most widely adopted open-source solution, LMCache [6], implements CacheBlend [40] through intrusive modifications to the underlying LLM execution flow. However, such structural changes disrupt the system-level optimizations intrinsic to modern inference engines (e.g., CUDA graphs and highly optimized attention kernels). This not only complicates GPU memory management but also severely degrades system flexibility. Moreover, to comply with the scheduling constraints of the inference engine, KV cache reuse is typically required to be contiguous from the beginning of the prompt. For instance, given a pool of cached chunks $\{A, B, C\}$ and a target prompt sequence $[B, A, D, C]$, reuse is strictly limited to the continuous prefix $[B, A]$. The isolated cached chunk C cannot be reused, as the un-cached chunk D breaks the continuity. This significantly limits reuse opportunities and leads to compromised overall efficiency.

To overcome the aforementioned limitations, this paper aims to **design a lightweight, high-quality, and position-independent KV cache reuse system that enables flexible reuse without disrupting the underlying LLM execution flow.** To this end, we introduce DARIC, a system-algorithm co-designed KV cache reuse method. At the algorithmic level, we propose a distribution-aware token selection strategy that adaptively schedules token recomputation based on consistent KV deviation patterns, striking an optimal balance between generation quality and computational efficiency (as illustrated in Figure 1 (right)). At the system level, DARIC features a lightweight architecture and is implemented on top of the modern LLM inference engine vLLM [23]. It enables independent KV cache generation and recomputation with minimal integration overhead, thereby fully preserving existing system-level optimizations. Figure 2 (right) shows that DARIC consistently achieves lower token selection and recomputation overhead compared to CacheBlend across different prompt lengths. In summary, we make the following contributions:

- We conduct a comprehensive analysis of KV cache deviation distributions across diverse models and datasets, uncovering consistent deviation patterns. Based on these observations, we develop a distribution-aware recomputation strategy that effectively balances generation quality and computational efficiency.
- We design a lightweight system architecture that seamlessly integrates DARIC into the widely adopted inference engine vLLM with minimal modifications. This architecture enables flexible, position-independent KV cache reuse while fully preserving its original scheduling semantics and system-level optimizations.
- We evaluate DARIC across various LLMs and application scenarios (e.g., RAG and agentic tool calling). Extensive experiments demonstrate that it consistently outperforms state-of-the-art methods, achieving a superior balance between generation quality and inference efficiency.

2 Preliminaries

2.1 Problem Definition

The goal of position-independent KV cache reuse is to independently pre-generate and store KV caches for each chunk of an input prompt, enabling their reuse in subsequent requests regardless of their absolute positions. Specifically, given an input prompt of L tokens, existing position-independent reuse systems [2, 15, 27, 28, 40] partition the prompt into a sequence of M chunks $[C_1, C_2, \ldots, C_M]$, where C_i denotes the i-th chunk containing L_i tokens, such that $\sum_{i=1}^{M} L_i = L$. Then, the independently pre-generated KV caches of these chunks are directly reused. The final KV cache $(\boldsymbol{K}, \boldsymbol{V})$ is obtained by simply concatenating the independent keys $[\boldsymbol{K}_i]_{i=1}^{M}$ and values $[\boldsymbol{V}_i]_{i=1}^{M}$ across all chunks. Recognizing that the independently generated KV caches may degrade generation quality due to the loss of inter-chunk correlations, existing methods[2, 15, 40] often adopt a partial recomputation strategy. Specifically, they select a small subset of token indices $\mathcal{I}$ and update their corresponding KV caches by recomputing them over the full, combined context.

2.2 Workflow

The workflow of partial recomputation-based position-independent KV cache reuse systems typically consists of the following steps:

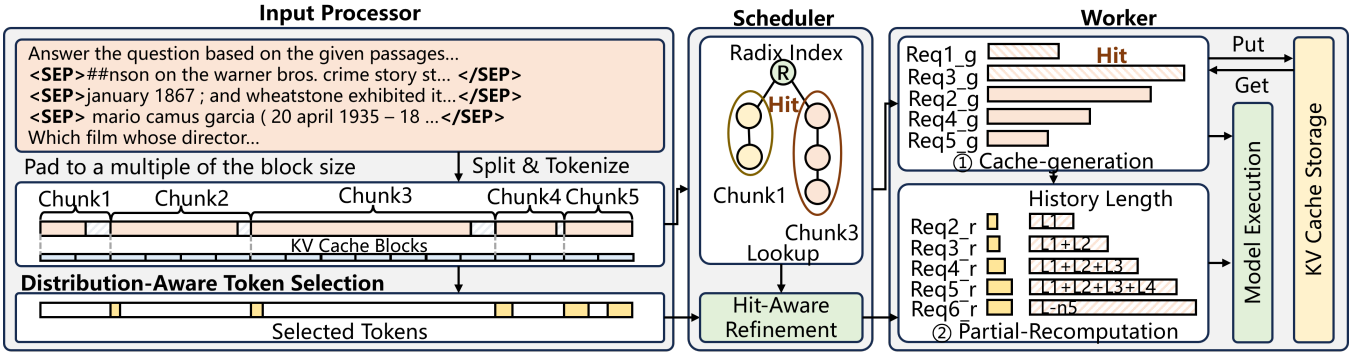


Figure 3: An overview of DARIC.

Hit status determination. To enable position-independent KV cache reuse, the system first determines the hit status of each chunk in the input prompt. In most existing methods [15, 27, 40], a cache hit occurs if an independently generated KV cache exists for the exact same text chunk. Some approaches [2] extend this by allowing standard prefix KV caches to be independently reused as long as they match the target chunk. Once all hits are identified, the corresponding KV caches are retrieved and assembled, yielding the initial concatenated KV cache $(\boldsymbol{K}, \boldsymbol{V})$.

Position re-encoding. The concatenated KV caches then undergo position re-encoding [27, 28] to align their positional information with the new context. Specifically, suppose the i-th chunk contains l_i tokens and originally started at position t_i. The re-encoding process remaps the absolute positions of the concatenated keys from their original, potentially disjoint indices $[t_1, \ldots, t_1 + l_1, t_2, \ldots, t_2 + l_2, \ldots, t_M, \ldots, t_M + l_M]$ to a contiguous sequence $[0, 1, \ldots, \sum_{i=1}^{M} l_i - 1]$. This remapping is mainly conducted on the rotary positional encoding (RoPE) [33] by applying a rotation matrix to the original keys, while the values remain unchanged during this process. This yields the re-encoded KV cache $(\boldsymbol{K}', \boldsymbol{V})$.

Recomputation token selection. A small subset of tokens is then selected for recomputation to recover cross-chunk correlations. Existing selection strategies generally fall into two categories: (1) online deviation-based approaches[2, 40], which dynamically identify tokens by computing the real-time deviation between independent and prefix-based KV caches; and (2) prior-driven methods[15], which rely on static, predefined rules derived from general architectural heuristics (e.g., attention sink). This yields a final index set $\mathcal{I} = \{i_1, i_2, \ldots, i_n\}$ for subsequent recomputation.

Partial KV Cache Recomputation. Finally, the KV caches for the selected tokens are updated by recomputing their key-value pairs using the full context [40]. Specifically, the required hidden states $\boldsymbol{H}$ are gathered from the full input sequence based on the target index set $\mathcal{I}$. The new queries $\boldsymbol{Q}^s$, keys $\boldsymbol{K}^s$, and values $\boldsymbol{V}^s$ are then computed directly from $\boldsymbol{H}$. To restore cross-chunk correlations, the newly generated $\boldsymbol{K}^s$ and $\boldsymbol{V}^s$ overwrite their corresponding entries in the position-re-encoded cache $(\boldsymbol{K}', \boldsymbol{V})$, yielding the fully updated KV caches $\boldsymbol{K}^u$ and $\boldsymbol{V}^u$. The final attention output for these tokens is subsequently formulated as:

$$\boldsymbol{O} = \text{softmax}\left(\frac{\boldsymbol{Q}^s(\boldsymbol{K}^u)^\top}{\sqrt{d}}\right)\boldsymbol{V}^u. \tag{1}$$

In existing systems (e.g., LMCache [6]), hit status determination is typically managed by the engine's scheduler to calculate the total hit length of the prompt. Meanwhile, the remaining steps are executed in the GPU workers by modifying the original LLM execution flow. Nevertheless, since the selected token set typically constitutes less than 15% of the original prompt, this partial recomputation is significantly faster than a full prefill pass.

3 DARIC System

In this section, we first provide an overview of the DARIC system. We then describe the distribution-aware token selection algorithm and the system-level design in detail.

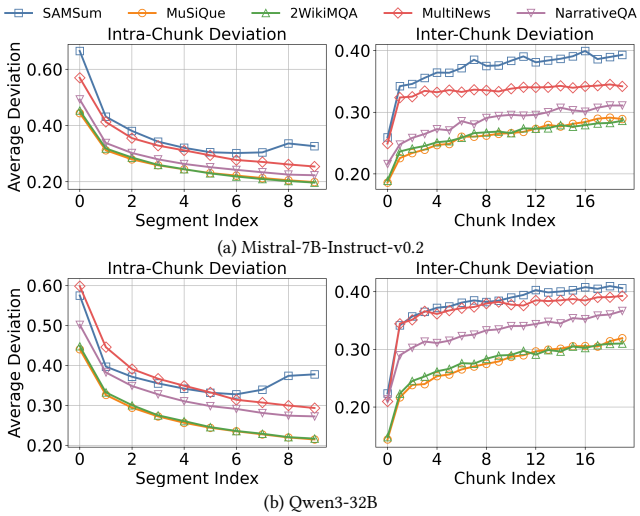
3.1 Overview

Figure 3 illustrates the overall architecture of the DARIC system. Our primary design goal is to integrate DARIC into existing inference engines with minimal modifications, fully preserving their original scheduling semantics and system-level optimizations. To achieve this, DARIC adopts the same high-level architecture as standard LLM inference engines (e.g., vLLM [23]), comprising three core components: the input processor, the scheduler, and the worker.

Input processor. Upon receiving a prompt, the input processor performs lightweight preprocessing without altering the inference engine's inherent scheduling logic. Specifically, it parses user-defined delimiters to identify chunk boundaries and applies necessary padding, ensuring chunk lengths strictly align with the block-size multiples required by the underlying attention kernels. Leveraging this chunk-level layout, the processor then executes the distribution-aware token selection algorithm to determine an initial index set of tokens for recomputation.

Scheduler. The scheduler operates identically to standard inference engines, determining the cache hit status of the input prompt, including both GPU prefix hits and externally offloaded non-prefix hits. Based on this hit status, the scheduler further refines the recomputation index set without modifying the original request structure or scheduling flow.

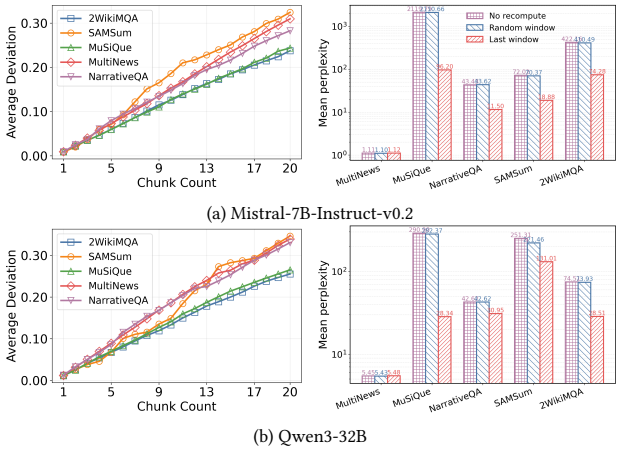
Worker. The prompt is decomposed into executable sub-requests once it reaches the worker. The worker handles the request via two distinct phases: a *cache-generation phase* that computes independent KV caches for newly arrived chunks, and a *partial-recomputation phase* to update the selected tokens. Crucially, both phases are



(a) Mistral-7B-Instruct-v0.2

(b) Qwen3-32B

Figure 4: Analysis of the intra- and inter-chunk KV deviation distribution.



(a) Mistral-7B-Instruct-v0.2

(b) Qwen3-32B

Figure 5: Effects of the number of independent chunks on average KV deviation and of latest-window recomputation on first-token perplexity.

executed by dispatching standard sub-requests directly to the existing model execution pipeline, avoiding the need for any custom execution paths.

Ultimately, these minimally intrusive designs empower DARIC to seamlessly integrate with modern inference engines while fully preserving their highly optimized scheduling, execution, and memory management mechanisms.

3.2 Distribution-Aware Token Selection

3.2.1 Observations. It is well-established that the deviation between independently generated and prefix-based KV caches fundamentally limits generation quality in position-independent reuse systems [40]. Therefore, we conduct an extensive empirical study on KV deviation distributions across various models and datasets to uncover consistent underlying patterns that can inform a highly efficient, prior-driven token selection strategy.

We quantify the KV deviation for the i-th chunk by calculating the normalized L2 distance between its independently generated and true prefix KV caches. This is formulated as:

$$\Delta(i) = \text{Mean}\left(\left\|\mathcal{N}(\boldsymbol{K}_i') - \mathcal{N}(\boldsymbol{K}_i^f)\right\|_2 + \left\|\mathcal{N}(\boldsymbol{V}_i) - \mathcal{N}(\boldsymbol{V}_i^f)\right\|_2\right), \quad (2)$$

where $\mathcal{N}(\boldsymbol{x}) = \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2}$ denotes the L2 normalization operation. $\boldsymbol{K}_i'$ and $\boldsymbol{V}_i$ represent the independent KV caches after position re-encoding, whereas $\boldsymbol{K}_i^f$ and $\boldsymbol{V}_i^f$ correspond to the standard prefix KV caches. We conduct experiments on two LLMs, Mistral-7B-v0.2 [18] and Qwen3-32B [37], using five datasets from LongBench [3]. The details are as follows.

Intra-chunk KV deviation. To analyze the distribution of KV deviation within each chunk, we randomly select M_t chunks from the datasets to construct the input context. We then divide every KV cache chunk into K equal-sized segments and compute the average KV deviation for each segment:

$$\Delta_{\text{intra}}(j) = \frac{1}{M_t}\sum_{i=1}^{M_t}\Delta(i,j), \quad (3)$$

where $\Delta(i,j)$ represents the KV deviation of the j-th segment in the i-th chunk, calculated via Equation 2. We repeat this sampling process N times and report the mean KV deviation. As depicted in the left column of Figure 4, this analysis leads to the following observation:

Observation 1. *The intra-chunk KV deviation consistently decreases as the segment index increases.*

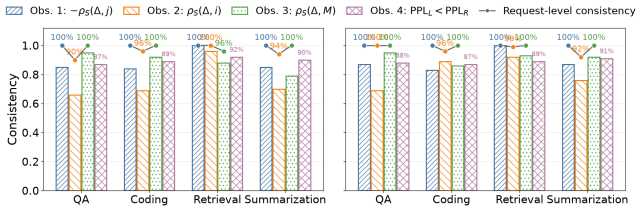
This pattern is also consistent with the attention-sink effect considered in EPIC [15]: chunk-initial tokens heavily influence subsequent attention computations, making their recomputation especially valuable. Consequently, prioritizing the recomputation of initial tokens within each chunk is a highly effective strategy to suppress the deviation of independent KV caches.

Inter-chunk KV deviation. To examine the deviation distribution across different chunks, we follow a similar sampling protocol, randomly selecting M_t chunks each time. We define $\Delta_{\text{inter}}(i) = \Delta(i)$ as the mean KV deviation of the i-th chunk. As shown in the right column of Figure 4, this analysis yields the following observation:

Observation 2. *The inter-chunk KV deviation steadily increases alongside the chunk index.*

Theoretically, for a token in chunk t, ideal attention incorporates keys from its own chunk as well as all preceding chunks, whereas an independently generated cache covers only the intra-chunk component: $\text{Attn}_t^{\text{full}} = \text{Attn}(\boldsymbol{q}_t, \mathcal{K}_{<t} \cup \mathcal{K}_t)$ and $\text{Attn}_t^{\text{ind}} = \text{Attn}(\boldsymbol{q}_t, \mathcal{K}_t)$, where $\mathcal{K}_{<t}$ denotes preceding chunks. The omitted inter-chunk component therefore grows with t. Thus, chunks positioned later in the prompt demand a proportionally higher recomputation rate to effectively mitigate the accumulated KV deviations.

Impact of independent chunk number. We further investigate how the total number of independent chunks impacts the



(a) Mistral-7B-Instruct-v0.2 (b) Qwen3-32B

Figure 6: Generalization of the four KV-deviation observations across workloads.

overall KV deviation. By merging adjacent chunks, we vary the number of independent chunks from 1 to 20 and compute the KV deviation for the entire prompt. As illustrated in the left column of Figure 5, this analysis leads to the following observation:

Observation 3. *The overall KV deviation steadily increases with the number of independent chunks.*

Similarly, more independent chunks result in more omitted inter-chunk interactions, causing KV cache deviation to accumulate across the prompt. Thus, highly fragmented prompts require a larger recomputation budget to reduce KV cache deviation.

Impact of the latest context window. As demonstrated by recent sparse attention methods [24, 26], the most recent tokens in a prompt are crucial for capturing critical recent context. Accordingly, we evaluate the impact of recomputing these latest tokens on generation quality. We compare the first-token perplexity under three settings: (1) no recomputation, (2) recomputing the most recent W tokens, and (3) recomputing a randomly positioned window of identical size (W tokens). As shown in the right column of Figure 5, this analysis leads to the following observation:

Observation 4. *The KV cache quality within the latest context window is critical to the overall generation quality.*

This observation is also consistent with the design of the aforementioned sparse attention methods [24, 26]. Therefore, recomputing the latest contextual window serves as an effective strategy for preserving end-to-end generation quality.

Generalization of Observations. These observations stem from the causal structure of self-attention and should therefore generalize across diverse workloads. We quantify their generality using tasks from LongBench [3] and InfiniteBench [43], grouped into four categories: QA (2WikiMQA, MuSiQue, NarrativeQA, and EN.QA), coding (LCC, RepoBench-P, Code.Debug, and Code.Run), retrieval (PassageRetrieval-en, Retrieve.PassKey, and Retrieve.Number), and summarization (MultiNews, SAMSum, and En.Sum). For each request, we compute the relevant Spearman correlations and report their median within each workload. As shown in Figure 6, the first three bars report the median direction-aligned correlations $-\rho_S(\Delta, j)$, $\rho_S(\Delta, i)$, and $\rho_S(\Delta, M)$, respectively, while the overlaid lines report the fraction of requests exhibiting the expected correlation direction. The fourth bar reports the fraction of requests whose first-token perplexity is lower with latest-window recomputation than with a same-sized random window. The results show that the median correlations remain strong across all workloads, consistently exceeding 0.6, with 87%–100% of individual requests following the expected trends. This indicates that these four observations capture broadly consistent properties of KV deviation rather than properties specific to particular tasks.

Algorithm 1: Token Selection

Input: number of chunks M, average chunk budget n_{avg}, ratio r, MTAF $f(x)$
Output: recomputation token index set $\mathcal{I}$ with $|\mathcal{I}| \le Mn_{\text{avg}}$

$[a, b, n_l]^T \leftarrow$ Solving Equation 4;
for $i \leftarrow 2$ **to** M **do**
  $n_i \leftarrow \lfloor af(i) + b \rfloor$;
  $\mathcal{I}_i \leftarrow \{\sum_{k=1}^{i-1} l_k + j\}_{j=1}^{n_i}$;
end
$\mathcal{I}_l \leftarrow \{\sum_{k=1}^{M} l_k - j\}_{j=1}^{n_l}$;
$\mathcal{I} \leftarrow \bigcup_{i=2}^{M} \mathcal{I}_i \cup \mathcal{I}_l$;
$iter \leftarrow 0$;
while $|\mathcal{I}| < Mn_{avg}$ **and** $iter < 3$ **do**
  $n_{\text{avg}} \leftarrow Mn_{\text{avg}} - |\mathcal{I}|$;
  if $n_{avg} > 0$ **then**
    Let $[s_{\text{avg}}, e_{\text{avg}}]$ be the last interval in $\mathcal{I}$;
    $\mathcal{I} \leftarrow \mathcal{I} \cup \{s_{\text{avg}} - k\}_{k=1}^{n_{\text{avg}}}$;
  end
  $iter \leftarrow iter + 1$;
end
return $\mathcal{I}$;

3.2.2 Token Selection Strategy. To align with the deviation patterns observed above, our token selection policy follows four main principles: (1) prioritize the first few tokens of each chunk; (2) scale the number of selected initial tokens proportionally to the chunk index; (3) adjust the total token budget according to the number of independent chunks; and (4) always include the latest context window to capture critical recent context.

Specifically, we define the total token budget as Mn_{avg}, where M is the total number of chunks and n_{avg} is the average token budget per chunk. Because the first chunk strictly matches the prefix and requires no recomputation, the total number of recomputed tokens is formulated as $n_{\text{total}} = \sum_{i=2}^{M} n_i + n_l$. Here, n_i denotes the recomputation budget for the initial tokens of the i-th chunk, and n_l denotes the budget for the latest context window. To mathematically instantiate our selection principles, we define two boundary conditions: $n_2 = rn_{\text{avg}}$ (where $r \in (0, 1)$) and $n_M = n_l$. This ensures the per-chunk allocation scales continuously up to n_l. We then model the allocation for intermediate chunks using a linear mapping $n_i = af(i) + b$. The term $f(i)$ represents a chosen Monotonic Token Allocation Function (MTAF), such as $f(x) = x$, $f(x) = e^x$, or $f(x) = \log(x)$. Ultimately, the variables a, b, and n_l can be solved subject to the total budget constraint $\sum_{i=2}^{M} n_i + n_l = Mn_{\text{avg}}$. This system of equations can be elegantly formulated in matrix form as $\boldsymbol{Ax} = \boldsymbol{c}$:

$$\begin{bmatrix} f(M) & 1 & -1 \\ f(2) & 1 & 0 \\ \sum_{i=2}^{M} f(i) & M-1 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ n_l \end{bmatrix} = \begin{bmatrix} 0 \\ rn_{\text{avg}} \\ Mn_{\text{avg}} \end{bmatrix}. \tag{4}$$

Therefore, the exact recomputation budget for each chunk can always be uniquely determined by computing $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{c}$.

Furthermore, while inter-chunk KV deviation consistently exhibits an increasing trend, its exact distribution shape varies significantly across different datasets and LLMs. Therefore, adaptively configuring the MTAF and the initial ratio r is crucial for optimal performance. We consider three representative MTAFs: the linear function $f(x) = x$, the logarithmic function $f(x) = \log(x)$, and the exponential function $f(x) = e^x$. The linear MTAF scales the token allocation at a constant rate. Conversely, the non-linear functions skew the budget distribution: the logarithmic function yields a more moderated increase, whereas the exponential function aggressively concentrates the token budget on the latest chunks. In practical scenarios, we adopt the linear function with $r = 0.1$ as the default configuration because it delivers stable performance across workloads without requiring manual tuning. As empirical data accumulates, an adaptive policy can be introduced. Specifically, let $\theta = (f, r)$ denote the selection configuration. We select θ via grid search over a candidate set to minimize the expected quality loss, approximated by the first-token perplexity gap relative to full recomputation, under a fixed recomputation budget.

3.3 System Design

3.3.1 Input Processor. In this stage, we mainly aim to parse and structure the prompt. The input prompt is first tokenized and then parsed based on user-defined chunk delimiters to determine the chunk boundaries. Subsequently, dynamic padding is applied to guarantee that the start of every chunk strictly aligns with the boundary of a physical KV cache block in the underlying LLM inference engine. Specifically, the required padding length p_i for the i-th chunk is calculated as $p_i = (b - (t_i \bmod b)) \bmod b$, where b is the configured block size and t_i is the original starting token index of the chunk. This block-alignment strategy confers two major architectural benefits: (1) it enables the construction of independent KV caches for individual chunks leveraging unmodified, highly optimized PagedAttention kernels; and (2) it inherently eliminates memory fragmentation across chunk boundaries, thereby facilitating more efficient KV cache I/O operations.

Subsequently, following the strategy detailed in Section 3.2, an initial token set (indexed by $\mathcal{I}_i$) is dynamically allocated for each chunk. Since these selected tokens may overlap across adjacent chunks, we first merge all overlapping sets into disjoint ones. As illustrated in Algorithm 1, to maintain the originally allocated recomputation budget, we compensate for any reduction in the total token count resulting from the merge operation. We achieve this by prepending the missing indices to the beginning of the last merged set. This merging and compensation process is repeated iteratively until either the total size of the sets matches the target recomputation budget Mn_{avg} or the maximum number of iterations is reached, ultimately yielding the final recomputation token index set $\mathcal{I}$.

3.3.2 Scheduler. The scheduler is primarily responsible for determining the hit status of the input prompt and dynamically refining the recomputation token set accordingly.

KV cache index. To facilitate efficient lookup of offloaded KV caches, we organize them via a customized radix tree structure. Each radix tree is anchored by a root node strictly tied to a specific model configuration and tensor-parallel (TP) size. The child nodes contain essential metadata of corresponding KV cache segments, including the sequence length, timestamps, and access frequencies (vital for subsequent cache eviction policies). Each node is uniquely identified via a prefix hash derived from its parent's ID and the exact token sequence of the current chunk. Leveraging the chunk boundaries parsed by the input processor, we further subdivide each chunk into finer-grained segments bounded by a configurable maximum length (e.g., 256 tokens). These segments serve as the atomic units for cache indexing and retrieval. Crucially, if a user submits a prompt without explicit chunk delimiters, the system treats the entire text as a single monolithic chunk. Under such conditions, our indexing mechanism naturally degenerates into standard, fixed-size prefix matching.

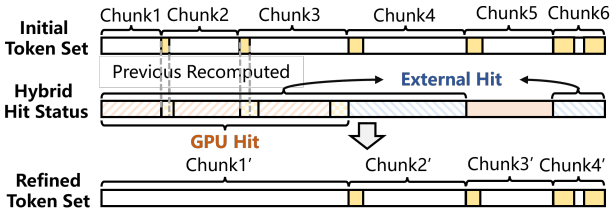


Figure 7: The hybrid cache-hit mechanism and hit-aware token set refinement.

Hybrid cache hit. As illustrated in Figure 7, DARIC supports a hybrid cache-hit mechanism that elegantly integrates native GPU prefix hits (provided by the inference engine) with externally retrieved KV cache hits. Given an incoming prompt, DARIC first queries the custom KV cache index to retrieve all externally hit chunks. Assuming the prompt is partitioned into an ordered sequence of chunks, DARIC classifies these hits based on their left-to-right continuity. Specifically, the *contiguous hit group* is defined as the longest contiguous prefix of the prompt where all inclusive chunks yield external hits. Conversely, the remaining externally hit chunks (e.g., isolated hits) constitute the *disjoint hit group*. In parallel, the native inference engine may report a resident GPU prefix hit of length L_{gpu}. Since the LLM engine schedules GPU computation strictly from the beginning of the prompt, only prefix-aligned hits are directly consumable. Therefore, DARIC dynamically determines the effective cache-hit length passed to the engine as $L_{\text{hit}} = \max\{L_{\text{gpu}}, L_{\text{cont}}\}$, where L_{cont} denotes the total token length covered by the contiguous hit group. Crucially, the externally hit within the disjoint hit group are not explicitly exposed to the engine's scheduler. Instead, they are transparently recorded and deferred to the worker side directly. This hybrid design enables DARIC to exploit additional KV cache reuse opportunities while remaining fully compatible with the existing prefix-based scheduling logic of highly optimized inference engines.

Hit-aware token set refinement. In hybrid cache-hit scenarios, the KV caches for prefix-aligned chunks are already managed and updated by the underlying inference engine. To prevent redundant computation, the target recomputation set must be dynamically adjusted to exclude tokens already residing in the GPU prefix. A subtle system challenge emerges during incremental prompt extensions, where new chunks are continuously appended. Under

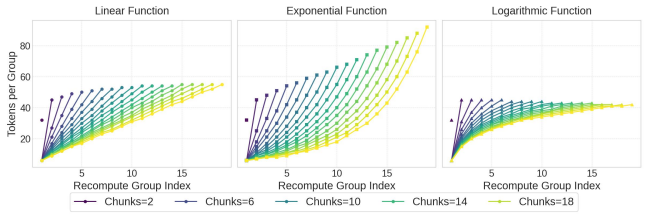


Figure 8: Selected token count under different MTAFs and total chunk counts.

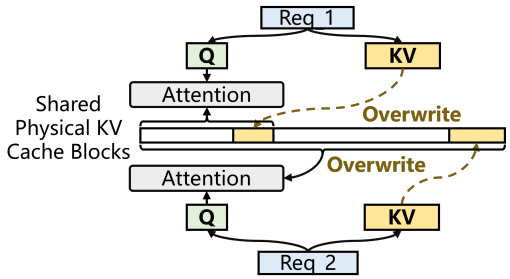


Figure 9: The attention operation in the partial recomputation phase.

dynamic chunk counts, the tokens recomputed in a prior step might conflict with the allocation required for the current prompt. DARIC resolves this via a built-in monotonicity property of its token allocation strategy: as the prompt grows, the algorithm shifts the recomputation budget toward newer chunks, guaranteeing that the recomputation requirement for any historically cached prefix chunk strictly never increases. We formalize this property as follows:

Proposition 1. *Let $f(i)$ be a strictly monotonic increasing function. For two chunk counts $(M < M')$, let $n_i^{(M)}$ and $n_i^{(M')}$ denote the recomputation token allocations derived from Eq. (4). Then for any $i \le M$, $n_i^{(M)} \ge n_i^{(M')}$.*

Figure 8 visualizes the number of selected tokens for each chunk under different chunk counts. Crucially, this monotonicity property allows DARIC to safely reuse the GPU prefix hits without needing to recompute any tokens within the prefix-aligned chunks, while still ensuring the recomputation budget for the remaining chunks is sufficient to mitigate KV deviations. Finally, the scheduler refines the recomputation token set by filtering out the tokens already covered by GPU prefix hits, yielding the refined token set $\hat{I}$ as well as chunk boundaries for the worker to process. In the inverse corner case where the current prompt is shorter, DARIC adopts a lazy acceptance strategy that also reuses the historical prefix cache directly, given that a cache sufficient for extended contexts naturally satisfies shorter ones.

3.3.3 Worker. The worker is fully responsible for driving the two core execution phases: cache generation and partial recomputation. Crucially, both phases are executed by dispatching specially formulated sub-requests to the engine's model execution pipeline.

Cache-generation phase. During this phase, each incoming scheduled request R is logically partitioned into a set of sub-requests

Algorithm 2: Partial Recomputation

Input: original request *req*, modified token indices $\hat{I}$, shared physical GPU KV cache blocks **C**
Output: updated KV cache **C** (equivalent to $\boldsymbol{K}^u, \boldsymbol{V}^u$)

Identify the number of modified chunks $\hat{M}$ via the scheduler;
Partition $\hat{I}$ into disjoint sets: $\hat{I} = \{\hat{I}^1, \hat{I}^2, \ldots, \hat{I}^{\hat{M}}\}$;
Divide *req* into corresponding sub-requests: $\mathcal{R}_r = \{r_r^1, r_r^2, \ldots, r_r^{\hat{M}}\}$;
Dispatch $\mathcal{R}_r$ as a batched request to the inference engine;
Gather input embeddings $\boldsymbol{E}$ for all tokens in $\hat{I}$;
Compute Queries $\boldsymbol{Q}^s$, Keys $\boldsymbol{K}^s$, and Values $\boldsymbol{V}^s$ concurrently using $\boldsymbol{E}$;
$\mathrm{C}[\hat{I}] \leftarrow (\boldsymbol{K}^s, \boldsymbol{V}^s)$;
foreach *sub-request* $r_r^j \in \mathcal{R}_r$ **in parallel do**
  $idx_end \leftarrow$ token position at the end of chunk j;
  Fetch updated prefix KV states $\boldsymbol{K}^u_{pref}, \boldsymbol{V}^u_{pref}$ from $\mathrm{C}[0 : idx_end]$;
  Let $\boldsymbol{Q}^j$ be the queries of sub-request r_r^j from $\boldsymbol{Q}^s$;
  $\boldsymbol{O}^j \leftarrow \mathrm{softmax}\left(\frac{\boldsymbol{Q}^j (\boldsymbol{K}^u_{pref})^T}{\sqrt{d}}\right) \boldsymbol{V}^u_{pref}$;
end
return C;

$\mathcal{R}_g = \{r_g^1, r_g^2, \ldots, r_g^{\hat{M}}\}$ corresponding to the modified chunk boundaries, where $\hat{M}$ is the modified chunk number. Each sub-request r_g^i takes exclusive responsibility for generating or loading the KV caches of its assigned chunk. Crucially, because the input processor guarantees block-aligned chunk boundaries, all $\hat{M}$ sub-requests can be seamlessly batched and executed concurrently in a single forward pass, fully leveraging native unmodified PagedAttention kernels. By decentralizing cache generation to this sub-request granularity, DARIC secures two distinct architectural advantages over existing systems like LMCache [6]. First, it strictly enables *independent reuse at arbitrary positions*, entirely breaking the rigid constraints of contiguous prefix matching. Second, it guarantees that the KV cache for each isolated chunk is computed in a perfectly prefix-free manner, thereby preventing any historical prefix context from polluting the chunk's semantic independence during subsequent reuse.

Partial-recomputation phase. After the cache-generation phase, the allocated GPU KV cache blocks are populated with initial KV states. To process the refined recomputation token set $\hat{I}$, we introduce a chunk-based in-place recomputation mechanism, as detailed in Algorithm 2. Instead of sequential processing, the selected tokens are partitioned into $\hat{M}$ chunk-aligned sub-requests $\mathcal{R}_r$. As illustrated in Figure 9, because all sub-requests share the original physical KV blocks, the newly projected $\boldsymbol{K}^s$ and $\boldsymbol{V}^s$ directly overwrite the stale cache values *in-place*. This physical overwrite inherently guarantees mathematical equivalence with the derivation of $\boldsymbol{K}^u$ and $\boldsymbol{V}^u$ in Equation 1. It naturally preserves strict causal correctness, as subsequent chunks seamlessly fetch the freshly updated KV prefixes from preceding chunks to compute their attention, circumventing any extra memory allocation. Ultimately, by dispatching $\mathcal{R}_r$ as a single

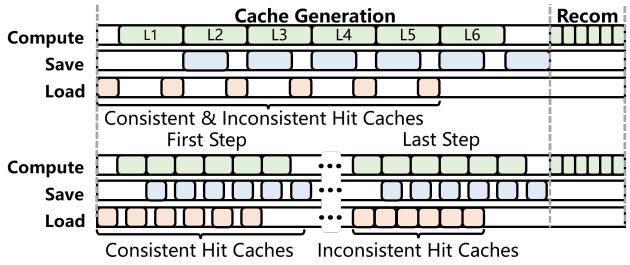


Figure 10: Worker-side pipeline of DARIC. Top: standard prefill; Bottom: chunked prefill.

batch, this design fully exploits the native execution optimizations of the inference engine, completing the entire partial-recomputation efficiently in a single forward pass.

Worker-side pipeline. We design an efficient worker-side pipeline that substantially enhances inference performance while strictly preserving the input/output interfaces of vLLM's original execution logic. As depicted in Figure 10, KV cache generation and partial recomputation are organized into two distinct stages. Crucially, not all requests must traverse both stages:

- The first stage is bypassed if all chunks yield a cache hit and can be directly loaded.
- The second stage is skipped if no tokens are selected for recomputation (e.g., when the prompt contains no independent chunks or is fully covered by the resident GPU prefix cache).

Because both stages involve complex KV cache I/O and intensive model execution, we design a highly parallelized execution flow that interleaves these operations. In the first stage, when uncached chunks are present, the worker dispatches cache-generation sub-requests $\mathcal{R}_g$ to compute independent KV caches for these chunks in a single forward pass. The cached chunks are loaded in parallel, and the newly generated KV caches are persisted to external storage concurrently. All these operations are conducted in a layer-wise manner. If the current request can be completed in a single step, DARIC directly loads all contiguous and disjoint KV caches. Otherwise, the worker loads only the contiguous caches, deferring the loading and reuse of disjoint hits to the final chunk-prefill step to ensure that all GPU KV cache blocks have been allocated. In the second stage, the worker processes the selected tokens for partial recomputation. The recomputation sub-requests $\mathcal{R}_r$ are dispatched as a single batch to the inference engine, which executes the in-place recomputation efficiently in a single forward pass.

4 Implementation

We implement DARIC on top of vLLM 0.11.0 (v1 engine) [23]. For the input processor, we develop a custom parser to identify chunk boundaries using user-defined delimiters and apply appropriate padding to ensure alignment with block-size multiples. To enable efficient management of offloaded KV caches, we design a KV cache index service based on radix trees backed by Redis, and a KV cache storage service that leverages both local disk and host memory. For disk offloading, KV caches are written to local storage via a pinned-memory buffer. For host-memory offloading, we directly allocate pinned memory to enable asynchronous, zero-copy data transfers between the GPU and host memory. In addition, we implement an optimized CUDA kernel to further improve data transfer efficiency between GPU memory and pinned host memory. Based on this efficient KV cache offloading management system, we introduce a custom *KV Connector* to enable the scheduler- and worker-side operations of DARIC:

Scheduler. Within the vLLM scheduler, we employ a Redis client to query the KV cache index service and determine the hit status of offloaded KV caches. To facilitate future cache management (e.g., LRU eviction), the index service subsequently updates the timestamps and reuse frequencies of the matched cache blocks. Based on these lookup results, we refine the recomputation token set by removing tokens that are already covered by GPU prefix hits.

Worker. We implement both the cache-generation and partial-recomputation phases by injecting a `rebuild_scheduler_output` function into the worker's model execution interface. This function splits the original request into multiple sub-requests and feeds them into the model runner. Regarding KV cache I/O operations, we submit the load tasks for all layers prior to model execution, waiting for the respective load to complete before executing each layer. Conversely, the save task for each layer is submitted immediately after the layer completes, enabling it to execute in parallel with the computation of the subsequent layer.

5 Experiments

5.1 Experiment Setups

5.1.1 Hardware and Model Configurations. All experiments are conducted on a single server equipped with four NVIDIA H20 GPUs (96 GB memory each). To support external KV cache offloading, we explicitly allocate 256 GB of host memory. We evaluate DARIC on three dense models: Mistral-7B-Instruct-v0.2 [18], Qwen3-32B [37], and Llama3.3-70B [34]. To assess its generalizability, we also evaluate two Mixture-of-Experts (MoE) models: Qwen3-30B-A3B [37] and MiniMax-M2 [30]. All models use standard FP16 precision. Specifically, Mistral-7B-Instruct-v0.2, Qwen3-32B, and Qwen3-30B-A3B are deployed with a tensor parallelism (TP) size of 1, Llama3.3-70B with TP = 2, and MiniMax-M2 with TP = 4. Across all experiments, the vLLM GPU memory utilization is fixed at 0.85.

5.1.2 Datasets and Workloads. We evaluate DARIC in three representative scenarios: (1) retrieval-augmented generation (RAG), (2) function calling, and (3) agentic coding. We also consider both synchronous and asynchronous workloads, covering controlled cache reuse analysis and realistic serving performance.

RAG scenario. Following [15, 40], we evaluate DARIC on five LongBench [3] datasets, including 2WikiMQA [13] and MuSiQue [35] (multi-document QA), SAMSum [10] (few-shot learning), MultiNews [9] (summarization), and NarrativeQA [22] (single-document QA). Each dataset contains 150–200 queries with up to 10 contexts per query. The 2WikiMQA, MuSiQue, and SAMSum datasets are adopted from [40], while MultiNews and NarrativeQA contexts are further segmented into 10 chunks of 256 and 512 tokens, respectively. As illustrated in Figure 11 (a), we treat each retrieved document natively as an independent chunk, while the fixed system prompt and the user query are treated as two distinct chunks.

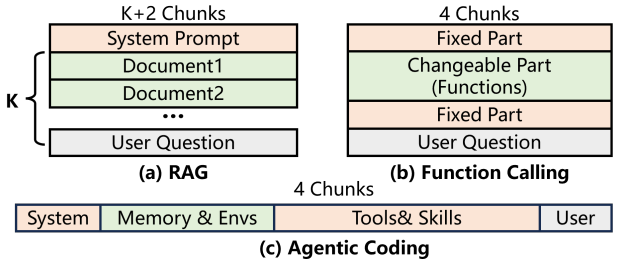


Figure 11: Different chunking strategies for RAG, function-calling, and agentic coding scenarios.

Function-calling scenario. We select the first 1,000 queries from the open-source dataset `openai-function-calling`[1] for evaluation. As illustrated in Figure 11 (b), this scenario presents a unique challenge: the embedded function descriptions dynamically vary across requests, whereas the surrounding system prompt remains static. To model this, we treat the dynamic function description as an isolated chunk, while the fixed text segments preceding and succeeding it are treated as separate chunks.

Agentic coding scenario. We also evaluate DARIC on SWE-bench [20] using the Claude Code agent. As illustrated in Figure 11(c), each prompt is partitioned into four chunks: two static components (the system prompt and tools/skills) and two request-dependent components (memory/environment context and the user request). This dataset is characterized by highly uneven chunk lengths, long output generations, and multi-turn tool interactions.

Workloads. We consider two distinct workloads: (1) *Synchronous workload*, where reusable prompt chunks (e.g., system prompts and retrieved documents in RAG, or fixed segments in function calling and agentic coding) are pre-generated and cached, and requests arrive sequentially to isolate the benefits of chunk-level KV cache reuse; and (2) *Asynchronous workload*, where requests with varying arrival rates are sent to a running vLLM service to measure end-to-end throughput and latency. To support these evaluations, we expand the 2WikiMQA dataset to 750 requests following [40]. This includes 600 LLM-generated queries using Qwen3-32B. For each query, we retrieve the top 6 relevant documents from all chunks in 2WikiMQA using vector search. The requests are shuffled, with the first 150 explicitly reserved for system warm-up.

5.1.3 Metrics. We evaluate DARIC in terms of both generation quality and efficiency. For generation quality, following the LongBench evaluation protocol, we use F1-score for 2WikiMQA, MuSiQue, and NarrativeQA to measure word-level overlap between model outputs and ground-truth answers, and Rouge-L for SAMSum and MultiNews to assess summary similarity based on the longest common subsequence. For the function-calling scenario, we use exact match (EM) and function name accuracy (FNA) to evaluate the correctness of the generated function calls, where EM requires an exact match of the entire function call string, while FNA only checks for the correctness of the function name, ignoring arguments. For efficiency, we report the time-to-first-token (TTFT) to measure the latency of producing the first output token. In addition, we measure the input throughput, defined as the number of input tokens processed per second, to evaluate the asynchronous performance under realistic workloads.

5.1.4 Baselines. We compare DARIC against four baselines:

- **Full Recompute**: LLM inference without any KV cache reuse during prefill.
- **Full Reuse**: Independent KV cache reuse without recomputation.
- **CacheBlend** [40]: Online-computation based independent KV cache reuse.
- **EPIC** [15]: Prior-driven token selection based independent KV cache reuse.

By default, we evaluate CacheBlend using its official implementation built on LMCache [6] with the vLLM V1 engine, adhering to its recommended 15% recomputation ratio. For EPIC, we implement its token selection strategy within our framework, configuring it with an equivalent recomputation budget (32 tokens per chunk)[2]. For DARIC, we adopt the default configuration across all scenarios (i.e., a linear MTAF with $r = 0.1$ and $n_{\text{avg}} = 32$).

5.2 Synchronous Performance

5.2.1 RAG scenario. We sweep the average per-chunk recomputation budget n_{avg} from 4 to 32 tokens for DARIC and EPIC, and from 5% to 15% of total tokens for CacheBlend. In addition, we observe that CacheBlend exhibits substantially degraded generation quality on Llama-3.3-70B. To ensure a competitive baseline, we extend its recomputation ratio to a higher range of 16%–32% for this model. Figure 12 compares the overall performance across five datasets and four models. DARIC's Pareto frontier generally lies close to the top-left corner, matching the quality of Full Recompute while retaining most of the latency benefits of Full Reuse. Across all evaluated configurations, DARIC achieves a 2.0×–8.2× speedup over Full Recompute. This acceleration becomes significantly more pronounced as the number of active parameters increases, scaling from 2.0× on 7B/MoE models to 8.2× on 32B/70B models. Furthermore, the efficiency gains are even more substantial on long-context datasets (e.g., NarrativeQA, MuSiQue, and 2WikiMQA), where speedups reach up to 8.2×, compared to 2.0×–3.5× on shorter summarization tasks (e.g., SAMSum and MultiNews). These results show that DARIC's quality–latency benefits generalize across RAG datasets and model scales. In most scenarios, the default $n_{\text{avg}} = 32$ is sufficient to maintain generation quality. For Mistral-7B on 2WikiMQA, the relatively large quality gap relative to Full Recompute can be narrowed by increasing n_{avg}. This gap likely stems from the weaker correlation between inter-chunk deviation and chunk index, and is further influenced by model capability and task complexity. Hence, dynamically adapting the recomputation budget is an important direction for future research.

Furthermore, compared to existing recomputation baselines such as EPIC and CacheBlend, DARIC achieves a superior quality–latency trade-off across all budget settings. While EPIC delivers the same speedup as DARIC, it suffers from substantial quality degradation.

[1] https://huggingface.co/datasets/madroid/glaive-function-calling-openai

[2] The EPIC configuration used here differs slightly from that in the original paper. Given M chunks, our total recomputation budget is $32M$, whereas the original paper uses $32(M-1)$.

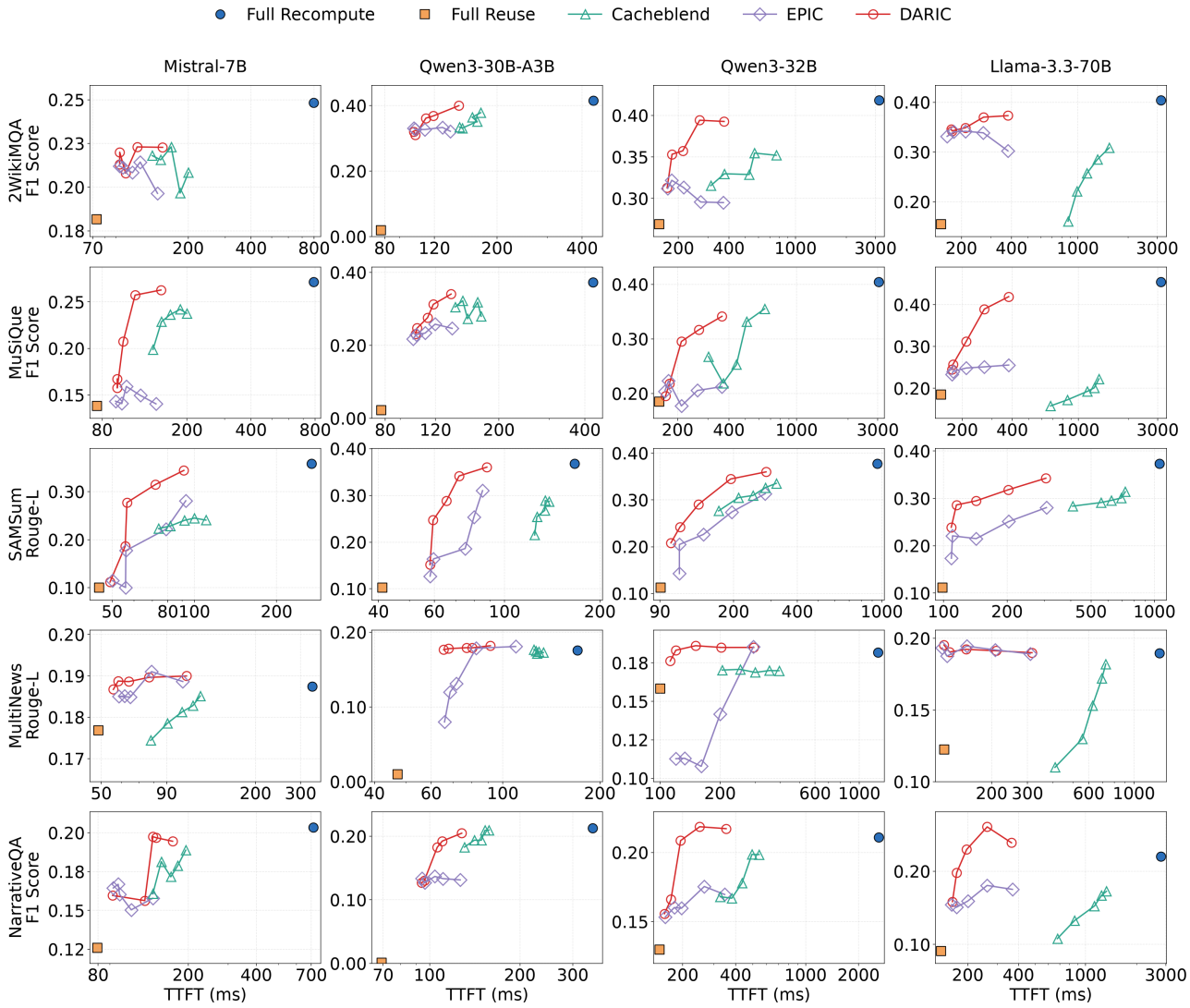


Figure 12: Comparison of DARIC and baseline methods under RAG scenario.

Conversely, although CacheBlend can attain comparable quality, it requires significantly more recomputation, leading to higher TTFT. These advantages stem from both a superior token selection strategy and lower execution overhead. First, unlike CacheBlend, which incurs runtime deviation computation overhead and relies on KV-deviation heuristics that overlook critical low-deviation local tokens, DARIC leverages structural attention priors to select high-impact tokens with zero online computation overhead. Second, unlike EPIC, which naively allocates recomputation budgets uniformly across chunks, DARIC adopts a distribution-aware strategy that accounts for monotonic inter-chunk error propagation, thereby preserving generation fidelity under identical budgets. Together, these results support the effectiveness of DARIC's design.

5.2.2 Function-calling scenario. We evaluate DARIC and the baselines on the function-calling dataset using Qwen3-32B, a model with strong native support for function calling. Considering that the context lengths of queries in this dataset are relatively short (200–300 tokens), we set the n_{avg} of DARIC and EPIC to 8 tokens per chunk, which occupies less than 15% of the total tokens in each chunk, aligning with the recommended budget of CacheBlend. As shown in Table 1, compared with full recompute, DARIC greatly retains the generation quality with only a 0.5% EM degradation and a 1.8% FNA degradation. Compared with CacheBlend, DARIC can also select function as well as arguments more accurately. Notably,

Table 1: Comparison of DARIC and baseline methods under function-calling scenario.

	Full Recompute	Full Reuse	CacheBlend	EPIC	DARIC
EM	86.90%	71.30%	86.20%	77.20%	86.40%
FNA	99.80%	81.40%	96.90%	87.30%	98.00%

Table 2: Comparison in the agentic coding scenario using MiniMax-M2 on SWE-bench.

	Full Recompute	Full Reuse	CacheBlend	EPIC	DARIC
Pass	68.60%	44.00%	68.20%	56.40%	68.40%
TTFT	2923.03 ms	1138.26 ms	2415.50 ms	1401.72 ms	1417.39 ms

Table 3: Generalization results on extended scenarios.

	2WikiMQA-M		LCC	
Method	F1 Score	TTFT	Edit Sim	TTFT
Full Recompute	0.4095	3210.98 ms	0.6608	1569.27 ms
Full Reuse	0.1110	135.22 ms	0.1725	94.70 ms
CacheBlend	0.3600	706.69 ms	0.6249	364.82 ms
EPIC	0.3826	369.44 ms	0.6159	301.77 ms
DARIC	0.4030	370.65 ms	0.6533	305.35 ms

EPIC suffers from severe quality degradation in this scenario, which can be attributed to its static token selection strategy that fails to capture the highly dynamic and sensitive nature of function-calling contexts. In contrast, DARIC's distribution-aware token selection effectively identifies critical tokens for recomputation, resulting in significantly improved generation quality as well as maintaining comparable efficiency. Beyond generation quality, evaluation on this scenario is crucial as it captures a prevalent tool-use pattern: variable function descriptions are interleaved between reusable fixed prompt segments, disrupting strict prefix continuity and necessitating flexible PIC reuse. In this setting, DARIC's system-level optimizations provide substantial additional gains by improving the cache hit ratio, as detailed in Section 5.3.3.

5.2.3 Agentic-coding scenario. We evaluate DARIC on the first 500 samples from the SWE-bench test set [20] using the Claude Code agent with MiniMax-M2. Experiments are conducted on vLLM 0.11.1, which natively supports this model architecture. As shown in Table 2, DARIC closely matches the pass rate of Full Recompute while achieving a 2.1× TTFT speedup. Compared to EPIC, DARIC delivers a significantly higher pass rate with comparable latency. Furthermore, compared to CacheBlend, DARIC maintains a comparable pass rate while providing a 1.7× speedup. These results demonstrate that DARIC effectively preserves end-to-end task success while substantially reducing prefill latency in realistic agentic coding scenarios.

5.2.4 Generalization to Extended Scenarios. To evaluate the robustness of DARIC's distribution-aware token selection, we conduct experiments on two extended datasets: 2WikiMQA-M and the LCC code completion dataset [11]. 2WikiMQA-M is constructed by appending a long output-formatting instruction to each query in 2WikiMQA, while for LCC (sourced from LongBench), each code context is partitioned into ten chunks. As shown in Table 3, DARIC preserves generation quality across both datasets while achieving the optimal trade-off between accuracy and efficiency. The results on long-formatting suffixes demonstrate that retaining the latest context window remains beneficial even when that region consists largely of repetitive formatting instructions, while the LCC results validate DARIC's ability to capture cross-chunk dependencies in non-RAG tasks. This consistency across diverse scenarios

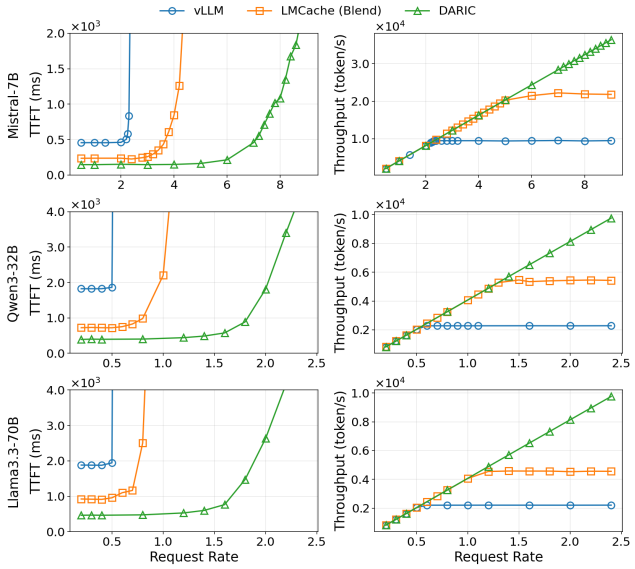


Figure 13: Comparison of asynchronous performance of different methods.

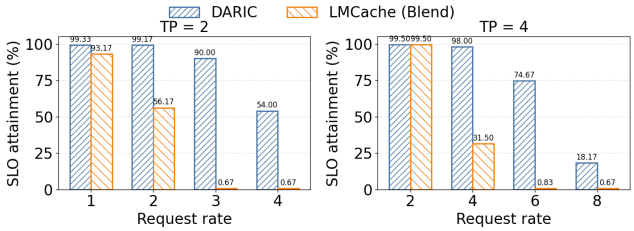


Figure 14: Comparison of SLO attainment ratios between DARIC and LMCache.

highlights the broad generalizability of DARIC's token selection, confirming that its effectiveness stems from the fundamental structural properties of causal attention.

5.2.5 Discussion: Adaptive Recomputation Budget. Our results show that the recomputation budget depends heavily on both the model and workload. While a relatively large n_{avg} is essential for complex multi-hop tasks (e.g., Mistral-7B on 2WikiMQA), a smaller value ($n_{\text{avg}} = 8$) suffices for short-chunk function calling. This motivates further research on request-level adaptive budgeting under two key principles. First, to balance quality and efficiency, combining CacheBlend's [40] ratio-based strategy with EPIC's [15] fixed-token scheme can cap the total budget Mn_{avg} at a fixed fraction of prompt tokens. Second, since online deviation computation disrupts the inference pipeline, combining offline deviation analysis with lightweight runtime monitoring provides an effective alternative (e.g., dynamically adjusting the budget via per-request attention distributions and offline deviation statistics on the deployed model).

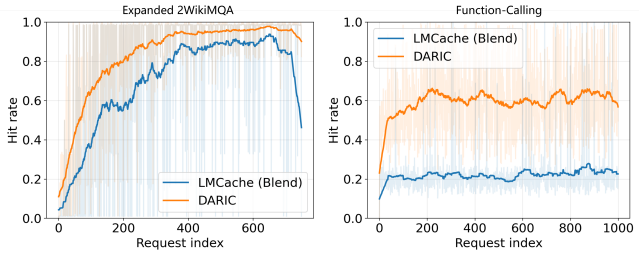


Figure 15: Comparison of hit rates between DARIC and LMCache.

5.3 Asynchronous Performance

5.3.1 Efficiency. We evaluate the end-to-end performance of DARIC and the baselines under asynchronous workloads with varying request arrival rates. We compare DARIC against the default vLLM implementation as well as vLLM augmented with LMCache to enable CacheBlend. For a fair comparison, LMCache and DARIC use the same host-memory offloading capacity of 256 GB. Figure 13 reports the TTFT and input throughput on the expanded 2WikiMQA dataset. We observe that DARIC achieves ~4× TTFT reduction over the default vLLM, and ~2× reduction over LMCache, demonstrating its efficiency under realistic serving workloads.

5.3.2 Robustness. We further evaluate the robustness of DARIC's system design by comparing it with LMCache under different tensor parallelism configurations. DARIC is designed to seamlessly integrate with vLLM's built-in optimizations, enabling it to better adapt to changes in system parallelism. To quantify robustness, we adopt the SLO attainment ratio, defined as the percentage of requests that satisfy a predefined TTFT threshold of 1000ms. Figure 14 reports the SLO attainment ratios of DARIC and LMCache on the expanded 2WikiMQA dataset using Qwen3-32B with tensor parallelism sizes of 2 and 4. The results show that DARIC consistently achieves higher SLO attainment ratios than LMCache across all settings, demonstrating both the robustness of DARIC's design and its effectiveness in leveraging vLLM's system optimizations.

5.3.3 Hit Rate. We also evaluate the KV-cache hit rate of DARIC and LMCache under asynchronous workloads. Since DARIC supports position-independent KV-cache reuse at arbitrary locations, whereas LMCache only enables position-independent reuse from the beginning of the prompt, we report the dynamic hit rates of both methods on the expanded 2WikiMQA dataset as well as the function-calling dataset. As illustrated in Figure 15, DARIC achieves consistently higher hit rates than LMCache across both datasets, with 1.2x on the expanded 2WikiMQA dataset and 3x on the function-calling dataset, demonstrating its superior ability to exploit KV-cache reuse opportunities in real-world scenarios. Notably, in real-world agentic scenarios, system prompts are often long and interleaved with multiple changeable components, which substantially restricts KV-cache reuse opportunities for LMCache. In contrast, DARIC can effectively reuse unchanged prompt segments regardless of their positions, resulting in significantly higher hit rates and improved overall performance.

5.4 Sensitivity Analysis

We conduct sensitivity analysis to investigate the impact of different configuration choices in DARIC, including the token selection, offloading device, and recomputation budget.

5.4.1 Token Selection. We first evaluate the effectiveness of each component in DARIC's token selection strategy. We compare DARIC against four variants: "Random" (selecting M random windows), "w/o Latest" (omitting the latest window), "w/o Chunk" (omitting chunk-initial tokens), and "Equal Chunk" (augmenting EPIC with latest-window recomputation). All comparisons use the default DARIC setting without tuning, i.e., a linear MTAF with $r = 0.1$ and $n_{\text{avg}} = 32$. As shown in Figure 16, DARIC achieves the highest accuracy across all datasets and models. Specifically, removing chunk-initial tokens causes the most severe performance drop, highlighting their importance in maintaining long-range coherence, while omitting the latest window also degrades performance. Furthermore, DARIC consistently outperforms "Equal Chunk" by an average of 8.07%, confirming the advantage of distribution-aware selection over uniform allocation. Meanwhile, all variants exhibit nearly identical TTFT, demonstrating that DARIC achieves these quality gains with negligible latency overhead.

Moreover, as discussed in Section 3.2, the distribution of selected tokens in the input prompt is primarily governed by the MTAF and the hyperparameter r. We consider two approaches for choosing MTAF and r: (1) a default configuration with $r = 0.1$ and a linear MTAF, and (2) an adaptive configuration obtained via grid search on a small validation set for each dataset. Figure 17 shows that the default policy delivers consistently strong performance across all five datasets and three models without requiring task-specific tuning. Furthermore, the adaptive policy provides additional performance gains, validating both the generalizability of our default policy and the utility of adaptive allocation.

5.4.2 Offloading Device. We further evaluate the impact of different offloading devices on DARIC's performance, considering both host memory and local disk. As shown in Figure 18, we compare the TTFT of DARIC under the two offloading options across varying input batch sizes. The results indicate that host-memory offloading consistently achieves lower TTFT than local disk offloading across all batch sizes. This advantage becomes more pronounced at larger batch sizes. These results suggest that, under disk-based offloading, the primary bottleneck of KV cache reuse lies in the data transfer overhead between the GPU and local disk rather than in computation. Consequently, our findings motivate hierarchical offloading designs that prioritize host memory for KV cache storage, as well as the adoption of GPU Direct Storage (GDS) to further accelerate data transfer.

5.4.3 Recomputation Budget. Since DARIC selects tokens based on the number of chunks rather than a fixed proportion of the total tokens as in CacheBlend, we further evaluate its robustness under different chunking strategies. Specifically, we define a cluster as a group of adjacent chunks that are treated as a single independent chunk during token selection. Consequently, the cluster size determines both the number of independent chunks and the corresponding chunk boundaries used for recomputation. In this experiment, we vary the cluster size from 1 (i.e., no clustering) to

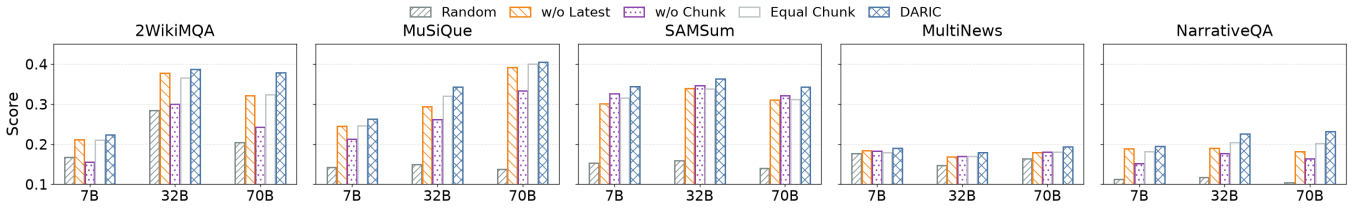


Figure 16: Comparison of different token selection strategies across multiple datasets and models.

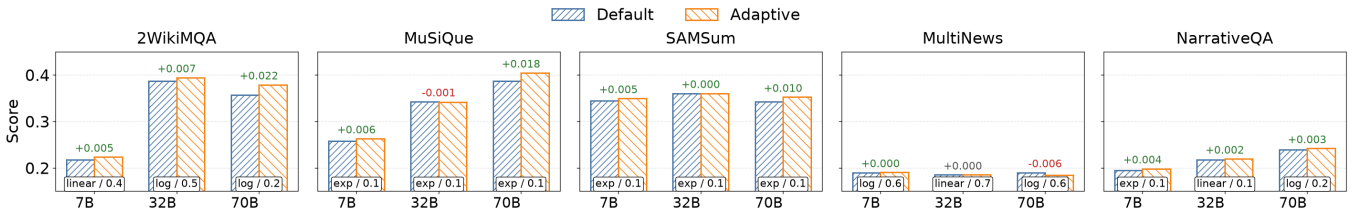


Figure 17: Comparison of DARIC under default and adaptive token selection policies. Numbers above the bars indicate score changes relative to the default policy, and boxes at the bottom denote the tuned (MTAF, r) configurations.

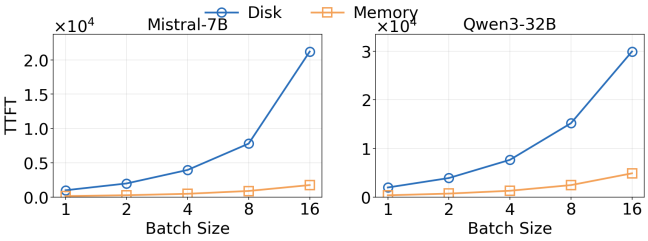


Figure 18: The TTFT of DARIC under different offloading devices.

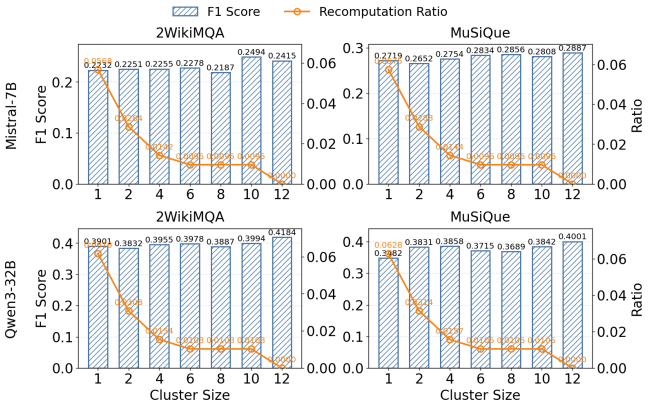


Figure 19: Generation quality and recomputation ratio of DARIC under different cluster sizes.

12 (i.e., full prefix reuse). Because the total recomputation budget is determined by the number of independent chunks, increasing the cluster size reduces the number of independent chunks and thus lowers the recomputation budget. As shown in Figure 19, the generation quality of DARIC remains relatively stable as the cluster size increases. These results indicate that, compared with CacheBlend's fixed-ratio token selection, DARIC can preserve comparable generation quality while significantly reducing the recomputation budget by strategically adjusting the chunking strategy.

6 Related Works

In this section, we first provide a brief overview of the KV cache in LLM inference. Then, we introduce two common KV cache reuse strategies: prefix reuse and position-independent reuse. Finally, we describe the recomputation-based cache adaptation workflow commonly adopted to support position-independent reuse.

6.1 KV Cache in Large Language Model

LLM inference mainly comprises two stages: prefill and decode [23]. The prefill stage processes the input prompt to generate the first token, while the decode stage generates subsequent tokens autoregressively. Considering that the self-attention mechanism [36] requires access to all previous token embeddings, processing long contexts is computation-intensive as well as memory-intensive. To accelerate inference, KV caches are widely used to store key and value tensors, thereby avoiding redundant computations. Noting the importance of KV caches in LLM inference, more and more recent works focus on the KV cache-centric acceleration in LLM inference, including quantization [14, 25], offloading [19, 32], management [23, 42, 44], and reuse [21, 27]. In this work, we focus primarily on KV cache reuse to accelerate the prefill stage.

6.2 KV Cache Reuse

Cross-request KV cache reuse accelerates the prefill stage by leveraging historical caches. Existing methods can be categorized into two major types according to their reuse strategies: prefix KV cache reuse and position-independent KV cache reuse.

Prefix KV cache reuse. Cross-request KV cache reuse accelerates the prefill stage by leveraging historical caches. Prefix-based methods reuse KV caches by matching the longest common prefix with historical requests [23, 44]. Most of these approaches use radix trees [44] to index and manage historical KV caches. To address GPU memory constraints, recent works have adopted hierarchical storage strategies utilizing DRAM and SSDs. For instance, RAGCache [21] implements a multilevel caching system tailored for RAG, while Mooncake [31] leverages disaggregated cluster resources to build a KV cache-centric system. However, these methods often suffer from low hit rates in scenarios like RAG, where prompts frequently differ across requests even though the underlying immutable knowledge chunks remain unchanged.

Position-independent KV cache reuse. Position-independent KV cache reuse enables flexible reuse by generating caches for each document independently [27, 28, 40]. While some methods fine-tune LLMs to adapt to these independent caches [27, 28, 38, 39], the high training cost limits their practical deployment. Consequently, recent work selectively recomputes reused KV caches to restore inter-document dependencies. For instance, CacheBlend [40] selects tokens based on cache deviation, whereas EPIC [15] relies on static, prior-driven rules. CacheCraft [2] further extends this strategy to prefix caches, ignoring their original positions in the full prompt.

7 Conclusion

In this paper, we propose DARIC, a system-algorithm co-designed position-independent KV cache reuse framework for efficient long-context LLM inference. DARIC enables flexible, position-independent cache reuse without altering the original LLM serving workflow. By leveraging a distribution-aware recomputation strategy, DARIC adaptively balances generation quality and recomputation overhead. Built on top of vLLM with minimal integration cost, DARIC fully preserves system-level optimizations while significantly expanding reuse opportunities. Extensive experiments across various models and scenarios demonstrate that DARIC consistently achieves a superior trade-off between generation quality and efficiency compared to state-of-the-art methods. An important direction for future research is to incorporate a request-adaptive budgeting mechanism into DARIC. Subject to a target TTFT or recomputation constraint, this mechanism would leverage request-specific signals to assess the marginal utility of recomputation, selectively assigning smaller budgets to simple requests and larger budgets to those experiencing severe KV deviations.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. In *Proceedings of the ACM on Management of Data*, Vol. 3. 1–28.

[3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2024. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. 3119–3137.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Proceedings of the Advances in Neural Information Processing Systems*.

[5] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of Artificial General Intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712* (2023).

[6] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. 2025. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. *arXiv preprint arXiv:2510.09665* (2025).

[7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In *Proceedings of the European Conference on Artificial Intelligence*. 2993–3000.

[8] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).

[9] Alexander Richard Fabbri, Irene Li, Tianwei She, Suyi Li, and Dragomir Radev. 2019. Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model. In *Proceedings of the annual meeting of the association for computational linguistics*. 1074–1084.

[10] Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. 2019. SAMSum corpus: A human-annotated dialogue dataset for abstractive summarization. In *Proceedings of the Workshop on New Frontiers in Summarization*. 70–79.

[11] Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian J. McAuley. 2023. LongCoder: A Long-Range Pre-trained Language Model for Code Completion. In *Proceedings of the International Conference on Machine Learning*, Vol. 202. 12098–12107.

[12] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and Fast Retrieval-Augmented Generation. In *Proceedings of the Findings of the Association for Computational Linguistics: EMNLP*. 10746–10761.

[13] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the International Conference on Computational Linguistics*. 6609–6625.

[14] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. In *Proceedings of the Advances in Neural Information Processing Systems*.

[15] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Hao Feng, Xusheng Chen, Yizhou Shan, Tao Xie, et al. 2025. EPIC: Efficient Position-Independent Caching for Serving Large Language Models. In *Proceedings of the International Conference on Machine Learning*.

[16] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. 2025. HiAgent: Hierarchical Working Memory Management for Solving Long-Horizon Agent Tasks with Large Language Model. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. 32779–32798.

[17] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 7036–7050.

[18] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[19] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2025. NEO: Saving GPU Memory Crisis with CPU Offloading for Online LLM Inference. In *Proceedings of the Conference on Machine Learning and Systems*.

[20] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *Proceedings of the International Conference on Learning Representations*.

[21] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. Ragcache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems* 44, 1 (2025), 1–27.

[22] Tomáš Kočiskỳ, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics* 6 (2018), 317–328.

[23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the Symposium on Operating Systems Principles*. 611–626.

[24] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM Knows What You are Looking for Before Generation. In *Proceedings of the Advances in Neural Information Processing Systems*.

[25] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Reza Haffari, and Bohan Zhuang. 2024. MiniCache: KV Cache Compression in Depth Dimension for Large Language Models. In *Proceedings of the Advances in Neural Information Processing Systems*.

[26] Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, et al. 2025. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In *Proceedings of the Advances in Neural Information Processing Systems*.

[27] Songshuo Lu, Hua Wang, Yutian Rong, Zhi Chen, and Yaohua Tang. 2025. Turborag: Accelerating retrieval-augmented generation with precomputed kv caches for chunked text. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 6599–6612.

[28] Dongyang Ma, Yan Wang, and Lan Tian. 2025. Block-attention for efficient prefilling. In *Proceedings of the International Conference on Learning Representations*.

[29] Shervin Minaee, Tomás Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large Language Models: A Survey. *arXiv preprint arXiv:2402.06196* (2024).

[30] MiniMax. 2026. The MiniMax-M2 Series: Mini Activations Unleashing Max Real-World Intelligence.

[31] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation - A KVCache-centric Architecture for Serving LLM Chatbot. In *Proceedings of the USENIX Conference on File and Storage Technologies*. 155–170.

[32] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the International Conference on Machine Learning*, Vol. 202. 31094–31116.

[33] Jianlin Su, Murtadha H. M. Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced transformer with Rotary Position Embedding. *Neurocomputing* 568 (2024), 127063.

[34] Llama Team. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024).

[35] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Proceedings of the Advances in Neural Information Processing Systems*. 5998–6008.

[37] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jian Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[38] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. 2025. KVLink: Accelerating Large Language Models via Efficient KV Cache Reuse. *arXiv preprint arXiv:2502.16002* (2025).

[39] Xinyu Yang, Tianqi Chen, and Beidi Chen. 2025. APE: Faster and Longer Context-Augmented Generation via Adaptive Parallel Encoding. In *Proceedings of the International Conference on Learning Representations*.

[40] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the European Conference on Computer Systems*. 94–109.

[41] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the International Conference on Learning Representations*.

[42] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of the Conference on Machine Learning and Systems*.

[43] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. ınftyBench: Extending Long Context Evaluation Beyond 100K Tokens. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. 15262–15277.

[44] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 37. 62557–62583.